Trabalho Prático — Jogo 2D Animado

Computação Gráfica

Atenção: O trabalho pode ser realizado **individualmente ou em dupla**. Entregue as atividades pelo Moodle. Todos os alunos poderão ser convocados aleatoriamente pelo professor para a **explicação e defesa do código entregue**, o que poderá afetar diretamente a nota final da atividade.

1 Objetivo

Desenvolver um jogo bidimensional interativo em Python, que pode ser estruturado em Orientação a Objetos (classes), com animação baseada no tempo. O programa deve ser construído expandindo a estrutura base do exemplo "Acerte o alvo" visto em aula, utilizando as bibliotecas `glfw, moderngl, numpy, pyglm` ou `pillow`.

O tema do jogo é livre. Algumas inspirações: *Arkanoid*, *Tetris*, *Geometry Dash*, *Pong*, *Space Invaders* e *Flappy Bird*. O jogo não precisa reproduzir o original. Uma versão simplificada, com as mecânicas centrais funcionando, é suficiente.

2 Especificações do Jogo

O programa deve cumprir, obrigatoriamente, os seguintes requisitos:

- **Animação Baseada no Tempo:** Todo movimento deve ser calculado a partir do intervalo de tempo Δt entre quadros, medido com a função `glfw.get_time()`, e as velocidades devem ser expressas em unidades por segundo. O jogo deve manter a mesma velocidade com o *vsync* ligado ou desligado. Para demonstrar isso, uma tecla deve ligar e desligar o *vsync* por meio da função `glfw.swap_interval`, e a taxa de quadros deve ser exibida na barra de título da janela.

- **Passo Máximo:** O valor de Δt deve ser limitado a um passo máximo, como no exemplo visto em aula, para evitar saltos quando a janela é arrastada ou o programa fica momentaneamente parado.

- **Animação Autônoma:** Além dos elementos controlados pelo jogador, pelo menos um elemento deve ter animação própria em função do tempo (por exemplo, oscilação, rotação, pulsação ou mudança periódica de cor).

- **Detecção de Colisão:** O jogo deve detectar colisões entre elementos (por exemplo, entre retângulos alinhados aos eixos, entre círculos ou entre círculo e retângulo) e reagir a elas (ricochete, destruição, pontuação, perda de vida etc.).

- **Entrada do Teclado:** Movimentos contínuos devem ser lidos por consulta direta, com a função `glfw.get_key`, e ações discretas (pausa, reinício, disparo etc.) devem ser tratadas por *callback* de teclado.

- **Estados do Jogo:** O jogo deve ter, no mínimo, os estados de início, em andamento, pausado e fim de jogo (derrota ou vitória), e deve permitir reiniciar a partida sem fechar o programa. Durante a pausa, nenhum elemento deve se mover, e ao retomar o jogo, o tempo pausado não deve ser acumulado em Δt.
- **Placar na Cena:** A pontuação e, se houver, o número de vidas devem ser exibidos na própria cena, desenhados com geometria (como as marcas de acerto do exemplo). Não é necessário renderizar texto.
- **Reaproveitamento de Geometrias:** Os VBOs devem ser criados apenas uma vez, na inicialização. A posição, a escala e a cor de cada elemento devem ser definidas por variáveis `uniform` ou por matrizes, sem reescrever os dados dos VBOs a cada quadro. Exceções devem ser justificadas no relatório.
- **Múltiplas Primitivas:** É obrigatório utilizar mais de um tipo de primitiva do OpenGL na construção dos elementos (por ex., misturar `moderngl.TRIANGLES` com `moderngl.TRIANGLE_FAN` ou `LINE_LOOP`).

3 Restrições Técnicas

- **OpenGL Core Profile:** O código deve utilizar estritamente o pipeline programável (OpenGL 4.0 Core). **É terminantemente proibido** o uso de recursos de OpenGL legado (*immediate mode*, `glBegin`, `glEnd`, matrizes embutidas, etc.).
- **Shaders:** A utilização de *shaders* personalizados é obrigatória. Toda a lógica de renderização e de cor deve vir dos arquivos `.vert` e `.frag` escritos em GLSL.
- **Bibliotecas Permitidas:** É permitido utilizar apenas o que já está instalado no *environment* inicial do curso (ex.: `moderngl`, `glfw`, `numpy`).
- **Proibição de Wrappers:** É proibido o uso de bibliotecas de alto nível que encapsulem o GLSL, o OpenGL ou a lógica de jogo (como Pygame, Pyglet, Arcade, PyOpenGL legado, motores de física, bibliotecas de cena prontas, etc.). O loop de jogo, a detecção de colisão e a animação devem ser implementados pelo grupo.
- **Exceções:** Caso o grupo deseje utilizar alguma biblioteca extra para um fim específico (ex.: ler um tipo de arquivo diferente ou tocar sons), deverá procurar o professor para validação e autorização prévias.

4 Entregáveis e Relatório

A entrega deverá ser submetida no Moodle (em formato `.zip`) contendo rigorosamente os seguintes itens:

1. **O Código-Fonte Completo:** Todos os arquivos necessários para rodar o projeto sem erros (`.py`, arquivos `.frag`, `.vert` e eventuais texturas/dados), com as instruções de execução e a lista de teclas no início do arquivo principal.
2. **Relatório do Projeto (PDF):** Um documento formal explicando o desenvolvimento, contendo:

- **Explicação da Arquitetura:** Explicação textual de como o código foi estruturado, como as classes se comunicam e como os *shaders* foram concebidos.
- **Laço de Jogo e Animação:** Explicação de como o Δt é medido e usado, de como a velocidade do jogo se mantém independente da taxa de quadros (com a taxa observada com o *vsync* ligado e desligado) e de como a pausa é tratada.
- **Colisões:** Descrição do teste de colisão utilizado, das formas geométricas envolvidas e de como o jogo evita que um objeto rápido atravesse outro sem colidir (tunelamento).
- **Screenshots:** Capturas de tela que evidenciem a execução do programa, mostrando claramente os estados de início, jogo em andamento, pausa e fim de jogo.
- **Divisão de Tarefas (se em dupla):** É obrigatório descrever detalhadamente a contribuição efetiva de cada um dos dois participantes na produção do código e do relatório.
- **Declaração de Uso de IA:** Caso a dupla/aluno tenha utilizado ferramentas de Inteligência Artificial (como GitHub Copilot, ChatGPT, Claude, etc.) como copiloto ou para auxiliar em alguma etapa de lógica ou de redação, **deve-se declarar explicitamente** qual ferramenta foi utilizada e para qual fim. Ressalta-se que, ao submeter o trabalho, o autor assume **total responsabilidade técnica e intelectual** pelo material (e eventuais bugs ou violações arquiteturais) gerado pela IA.